Implementation of Classical Optical Flow Estimation Techniques

Benjamin Luo
Department of Management Science and Engineering
University of Waterloo
Waterloo, Canada
b33luo@uwaterloo.ca

***Overview*—This report overviews the implementation and evaluation of the Lucas-Kanade and Horn-Schunck trackers via optical flow estimation.**

I. Methodology

A. Lucas-Kanade tracker

The Lucas-Kanade (LK) tracker is a localized algorithm that performs state estimation on key feature points in the frame instead of the entire frame. As a result, the optical flow vectors are sparse with more efficient computation.

The first step is extracting key feature points to initialize the algorithm. A strong feature is one that has high contrast and texture, which is typically a corner or edge. Two popular methods for feature identification are Shi-Tomasi and Harris. Both algorithms attempt to identify corners based on pixel intensities but with different underlying formulations. Both algorithms are implemented through OpenCV and compared below:

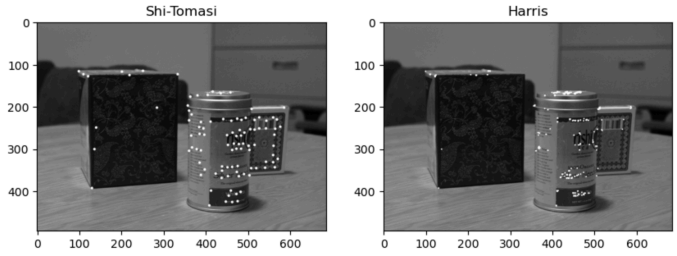


Fig. 1. Comparison of Shi-Tomasi and Harris feature detectors

After experimenting with hyperparameters in each feature detector, I proceed with the Shi-Tomasi method as it appears to identify stronger points based on the spacing and texture. 100 points are sampled with the Shi-Tomasi method as this number sufficiently covers all 3 foreground objects.

In addition to the input features, the LK method also requires 2 image frames and a window size. Analysis is done on window sizes of (3x3), (5x5), and (9x9) and adjacent frames in a video stream.

The LK method can be implemented through OpenCV's `calcOpticalFlowPyrLK` method where the high-level process entails:

(1) Forming a window around the feature point
(2) Warping the window using the current flow estimate
(3) Computing the gradient of the error wrt to flow
(4) Updating the flow estimate via the gradient
(5) Checking for convergence, otherwise repeating from step (2)

B. Lucas-Kanade coarse-to-fine tracker

Conventional optical flow estimation methods such as LK are designed for consecutive frames as per the small motion assumption. As a result, the LK method may deliver subpar results when presented with non-consecutive frames.

This motivates the use of coarse-to-fine techniques where the frame is analyzed at different resolution levels, starting from coarse (i.e. low resolution) and progressing toward finer (i.e. higher resolution) representations. This reduces the impact of large motions and improves the algorithm's robustness to noise.

A pyramidal structure can be formed where each level represents the frame at varying resolutions (figure 2). The LK method is applied at each level and uses the flow from coarser (i.e. higher) levels to estimate optical flow at finer levels. The number of layers and the scaling factor at each level are user-provided parameters but we assume a default of 3 levels and a scaling factor of 0.5.

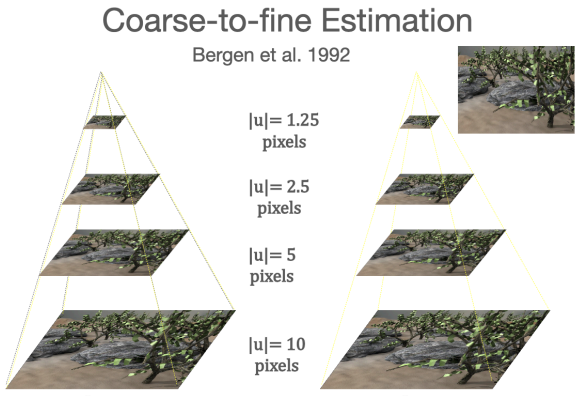


Fig. 2. Coarse-to-find pyramidal estimation (|u| represents the magnitude of the optical flow vector)

The high-level implementation of the pyramidal coarse-to-fine LK method is:

(1) Create the pyramid for the 2 image frames with varying resolutions
(2) Initialize flow at the coarsest level
(3) From coarse to fine:
    (a) Warp the next frame using the current flow estimate
    (b) Compute the optical flow at the current level
    (c) Update the flow estimate using the fine level flow
    (d) Upsample the flow to the finer level
(4) Return the final flow estimate

C. *Horn-Schunck tracker*

Unlike the LK method, the Horn-Schunck (HS) tracker is a global estimator so there is no need to perform feature identification. As a result, the optical flow vectors are dense and more computational than the LK optical flow vectors. The global estimation also enables smoother flow fields which is suitable for video streams with more movement.

The inputs to the HS method are 2 frames and a smoothness term (lambda) that penalizes large flow gradients. The algorithm is also based on a global energy function which is given in figure 3. Lambda values of 0.01, 0.1, and 1 are selected for comparison.

$$E = \sum \left[(I_x u + I_y v + I_t)^2 + \lambda^2 \left(\|\nabla u\|^2 + \|\nabla v\|^2 \right) \right]$$

Fig. 3. Energy equation used for HS optical flow estimation

The high-level implementation is:

(1) Initialize the flow field with zeros
(2) Compute the spatial gradient of the frames
(3) Compute the smoothness weights
(4) Iteratively compute flow updates and update the flow field until convergence

D. *Data and quantitative evaluation*

Evaluation is done on a video dataset of 3 objects on a table with a moving camera that totals 13 frames and 12 ground truth matrices. The dataset is loaded via OpenCV as grayscale numpy arrays of size (494, 685). Movement between adjacent frames is smooth and there is no background noise which makes the dataset suited for classical tracking methods.

Quantitative evaluation is done through endpoint error (EPE) and angular error (AE):

- **EPE**: Measures the euclidean distance between estimated and ground truth flow vectors (fig. 4)
- **AE**: Measures the angular displacement between estimated and ground truth flow vectors (fig. 4)

$$||GT - E||, \qquad \cos^{-1}\left(\frac{GT \cdot E}{||GT||\ ||E||}\right)$$

Fig. 4. EPE (left) and AE (right) formulas. GT=ground truth, E=estimate

E. *Qualitative evaluation*

Qualitative evaluation of trackers entails the empirical observation of various plots that display the optical flow vectors.

Quiver plots show a 2D representation of optical flow vectors and are especially suitable for visualizing the HS method as they can cleanly show entire optical flow fields (figure 5).

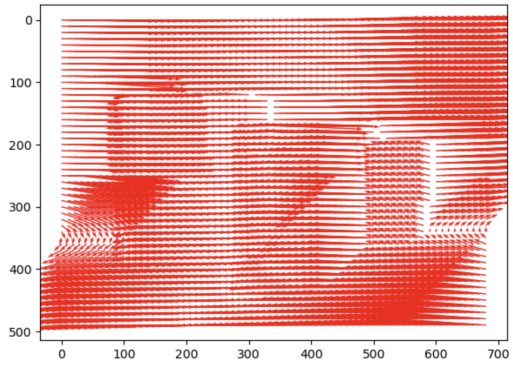


Fig. 5. Quiver plot of true optical flow from frame 1 → 2

Color-coded maps assign colors based on the magnitudes and directionality of the optical flow vectors where blue represents slow motion and red indicates fast motion (figure 6). The objects in the foreground are blue in this case because the camera is moving so further objects appear more animated.

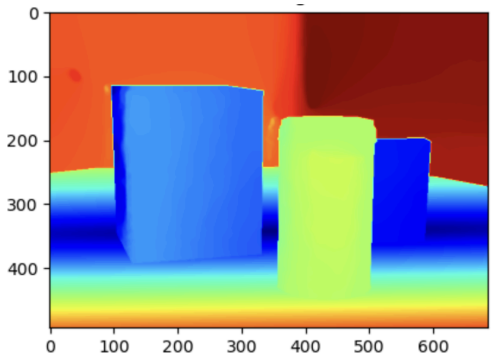


Fig. 6. Color-coded map of true optical flow from frame 1 → 2

Feature track overlays plot the trajectory of a set of feature points across frames. This is ideal for visualizing LK optical flow over multiple frames because the vectors are sparse and will not clutter the plot (figure 8). The below example shows only 1 frame of movement where the circles represent position in the next frame, lines represent optical flow vectors, green represents ground truth, and red represents estimates. The ground truth data is overlaid over the estimated data to make inconsistencies more visible.

II. Results

F. Lucas-Kanade tracker (Consecutive frames)

I implemented the LK tracker twice - once using the built-in CV2 method, and once from scratch as a sanity check. It goes without saying that the CV2 one performs better and in fact, the mathematical one visibly failed to perform well for the non-consecutive frame test. An excerpt of the results are displayed in figure 7, with full results available in appendix 1.

	0	1	2	3	4	5	6	7	8	9
frame	1-2	2-3	3-4	4-5	5-6	6-7	7-8	8-9	9-10	10-11
window_size	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)
epe	2.54	4.57	5.58	5.37	7.49	3.71	3.12	3.14	2.41	2.68
ae	25.57	41.24	42.83	33.61	32.26	27.11	37.58	34.19	35.63	21.87

	0	1	2	3	4	5	6	7	8	9
frame	1-2	2-3	3-4	4-5	5-6	6-7	7-8	8-9	9-10	10-11
window_size	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)	(3, 3)
epe	5.42	6.21	6.21	5.9	6.07	5.73	4.79	4.73	4.26	4.06
ae	111.86	110.3	112.69	123.45	117.1	123.46	121.71	125.17	120.39	114.41

Fig. 7. LK tracker performance with (3x3) window from frames 1-11. Top is the `cv2.calcOpticalFlowPyrLK` and bottom is the implementation from scratch

Figure 7 shows the ground truth optical flow vectors (green) overlaid over the estimated optical flow vectors (red).

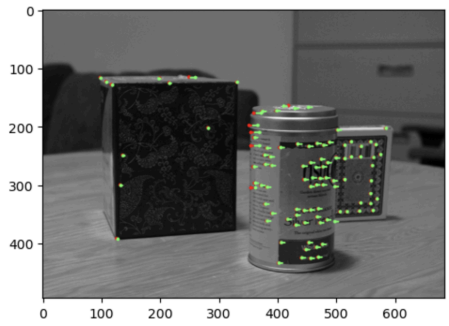


Fig. 8. Feature track overlay of true and estimated flow from frame 1 → 2

G. Lucas-Kanade tracker (Non-consecutive frames)

The LK implementation was tested between frames 1 and 13 to yield the following feature track overlay in figure 9:

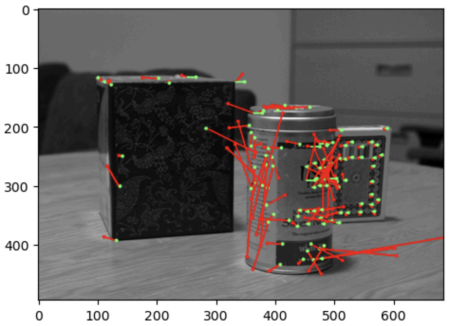


Fig. 9. Non-consecutive LK output from frame 1 → 13

I had to resort to using the CV2 LK implementation for figure 9 and subsequent analysis due to the inaccurate results from my own implementation (see appendix 2 for the results).

For quantitative assessment, it is not possible to compute EPE and AE as the ground truth data is provided for consecutive frames so I instead record the mean and standard deviation of the estimated optical flow vectors:

- Mean value: [3.08, 2.32]
- Standard deviation: [38.1, 42.55]

H. Coarse-to-fine Lucas-Kanade

Figure 10 shows the resultant quiver plot from the coarse-to-fine Lucas-Kanade. The runtime was 10.8 seconds to compute the optical flow from frame 1 → 13 whereas the runtime was negligible for the other LK implementations. The mean optical flow value is [0.3, -0.1] with standard deviation [0.53, 0.49]. Again, it is not possible to quantitatively assess this model due to the provided ground truth data being for consecutive frames.

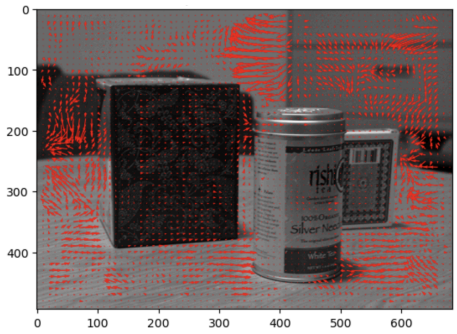


Fig. 10. Pyramidal coarse-to-flow Lucas Kanade from frame 1 → 13 with 3 layers and a scaling factor of 0.5.

I. Horn-Schunck

The implementation for this one was difficult (see appendix 4). When incorporating the energy function (figure 3), there would be many arithmetic and floating point errors triggered, likely due to the vanishing or exploding gradients. The output for frames 1 → 2 is given in figure 11 and evaluation metrics in figure 12 (the full data is available in appendix 5).

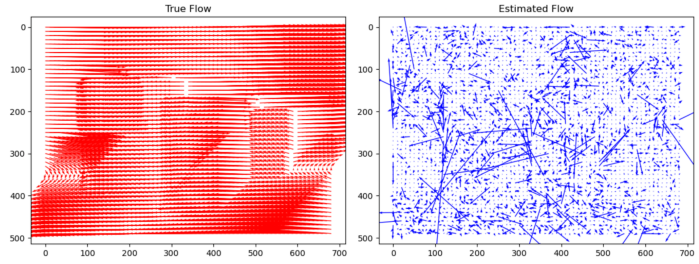


Fig. 11. Quiver plot of HS method from frame 1 → 2 with lambda = 1

The HS method had significantly higher runtime than LK with the single-frame cell taking 1.5 seconds to complete

compared to 0.1 seconds. All 13 frames together took 54.5 seconds to process compared to LK's 1-2 second range.

	0	1	2	3	4	5	6	7	8	9
frame	1-2	2-3	3-4	4-5	5-6	6-7	7-8	8-9	9-10	10-11
lambda	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01
epe	106.52	128.84	141.3	136.1	131.79	135.27	141.31	142.18	140.65	134.36
ae	87.03	87.53	87.31	87.36	88.24	87.47	87.39	87.53	87.81	87.28

Fig. 12. Excerpt of HS evaluation from frames 1 → 11 with lambda=0.01

III. Discussion

J. Lucas-Kanade tracker

As it turns out, `cv2.calcOpticalFlowPyrLK` has a built-in pyramid which may explain why it outperformed the LK implementation from scratch. The CV2 model had EPE in [2.41, 7.49] and AE in [21.87, 42.83] while the non-CV2 had EPE in [2.54, 6.21] and AE in [98.37, 125.16]. While the non-CV2 implementation had a reasonable EPE, the AE was unusually high; averaging around 120 degrees. This indicates the underlying code may not have been implemented correctly, especially given the non-consecutive flow estimation result in appendix 2.

In general, the LK method was highly efficient in computation, with individual frame-by-frame analysis completing almost instantly, and all 13 frames completing in about 1 second.

K. Horn-Schunck tracker

I cannot fairly assess the HS tracker as my implementation seems to be lacking in terms of qualitative assessment (figure 11). However, based on the results in appendix 5, the EPE is noticeably larger than the LK method, along with the AE (if only the CV2 LK is considered). This trend is consistent with past research findings (see appendix 6) but also greatly exaggerated given the EPE values that range from 100-140 despite the image dimensions only being (494, 685).

The evaluation metrics do not appear to vary between lambda values in the set {0.01, 0.1, 1}. There does appear to be a possible trend of the EPE worsening as the number of frames increases but this may be attributed to variable camera motions and scene movements between certain frames.

L. Non-consecutive Lucas-Kanade

Figures 9 and 10 show the vast disparity in the outputs between the conventional LK and coarse-to-fine pyramidal LK models for non-consecutive frames. The conventional LK uses corner points as an input whereas the coarse-to-fine LK uses points from lower resolution levels as input for estimating optical flow. This results in a denser optical flow field (figure 10) and consequently a much longer runtime given both the higher number of feature points and the additional computation from analyzing the frame at varying resolutions.

Looking further into figure 9, it appears the model struggles to estimate optical flow as the vectors have high variability despite motion being smooth. This shows in the high standard deviation of the optical flow vectors, especially when compared to the coarse-to-fine approach. Therefore, the LK model fails to perform well when the temporal coherence and small displacement assumptions are violated.

On the other hand, the coarse-to-fine model empirically performed far more accurately, but at significant cost to runtime. There were also some potential issues as seen in the foreground images as the optical flow should intuitively be uniform, yet there's noticeable differences in magnitude. Nonetheless, it still outperformed conventional LK and this is likely attributed to the coarse-to-fine LK's ability to observe motion at various resolutions which enables it to gain both a holistic and microscopic perspective.

Varying the depth of the pyramid and scaling factor significantly impacts the runtime but does not drastically change the model's ability to estimate optical flow as it still captures the high-level trend (appendix 3).

M. Lucas-Kanade v Horn-Schunck

The Lucas-Kanade is a localized algorithm with sparse optical flow vectors (figure 8) whereas the Horn-Schunck algorithm is global with dense optical flow vectors (figure 11). Their implementation and assumptions are similar, but Lucas-Kanade starts with a corner detection step which allows it to work with fewer points and thus achieve more efficient execution. In this case, it performed about 15x faster than the HS method.

In terms of performance, appendix 6 reveals that HS methods perform worse in terms of EPE and AE, but at the benefit of having far denser optical flow fields; making it ideal for tasks that are interested in observing more points of interest within an image, such as a moving background. The dense flow vectors are also reminiscent of the coarse-to-fine LK where information from coarser layers was propagated down to finer layers as feature points. As a result, the final output also appeared to be rather dense (figure 10); at a similar cost to runtime.

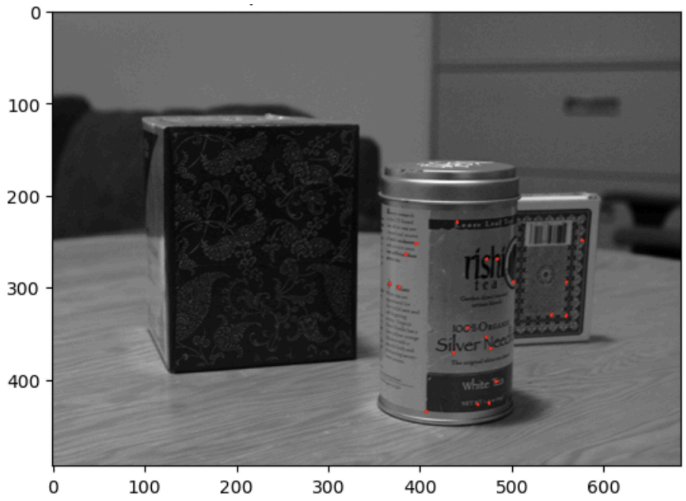
IV. Appendix

Appendix 1: Lucas-Kanade method on the video stream via the built-in CV2 implementation (left) and from scratch (right)

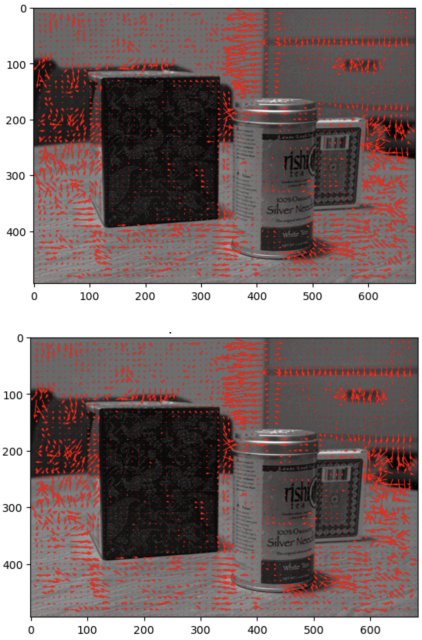
	frame	window_size	epe	ae
0	1-2	(3, 3)	2.54	25.57
1	2-3	(3, 3)	4.57	41.24
2	3-4	(3, 3)	5.58	42.83
3	4-5	(3, 3)	5.37	33.61
4	5-6	(3, 3)	7.49	32.26
5	6-7	(3, 3)	3.71	27.11
6	7-8	(3, 3)	3.12	37.58
7	8-9	(3, 3)	3.14	34.19
8	9-10	(3, 3)	2.41	35.63
9	10-11	(3, 3)	2.68	21.87
10	11-12	(3, 3)	2.87	18.99
11	12-13	(3, 3)	2.98	28.66
12	1-2	(5, 5)	1.91	19.95
13	2-3	(5, 5)	2.80	24.05
14	3-4	(5, 5)	3.90	31.59
15	4-5	(5, 5)	3.03	23.34
16	5-6	(5, 5)	2.69	23.43
17	6-7	(5, 5)	2.24	21.93
18	7-8	(5, 5)	1.99	35.91
19	8-9	(5, 5)	1.97	26.17
20	9-10	(5, 5)	1.51	32.75
21	10-11	(5, 5)	1.92	16.63
22	11-12	(5, 5)	1.33	11.64
23	12-13	(5, 5)	1.42	20.30
24	1-2	(9, 9)	1.43	17.07
25	2-3	(9, 9)	2.40	20.17
26	3-4	(9, 9)	3.22	26.94
27	4-5	(9, 9)	3.70	26.72
28	5-6	(9, 9)	3.42	20.97
29	6-7	(9, 9)	2.38	23.88
30	7-8	(9, 9)	1.39	31.85
31	8-9	(9, 9)	2.19	27.86
32	9-10	(9, 9)	1.47	31.79
33	10-11	(9, 9)	1.43	13.15
34	11-12	(9, 9)	1.64	14.45
35	12-13	(9, 9)	1.43	17.66

	frame	window_size	epe	ae
0	1-2	(3, 3)	5.42	111.860001
1	2-3	(3, 3)	6.21	110.300003
2	3-4	(3, 3)	6.21	112.690002
3	4-5	(3, 3)	5.90	123.449997
4	5-6	(3, 3)	6.07	117.099998
5	6-7	(3, 3)	5.73	123.459999
6	7-8	(3, 3)	4.79	121.709999
7	8-9	(3, 3)	4.73	125.169998
8	9-10	(3, 3)	4.26	120.389999
9	10-11	(3, 3)	4.06	114.410004
10	11-12	(3, 3)	3.13	98.370003
11	12-13	(3, 3)	2.54	118.279999
12	1-2	(5, 5)	5.42	111.860001
13	2-3	(5, 5)	6.21	110.300003
14	3-4	(5, 5)	6.21	112.690002
15	4-5	(5, 5)	5.90	123.449997
16	5-6	(5, 5)	6.07	117.099998
17	6-7	(5, 5)	5.73	123.459999
18	7-8	(5, 5)	4.79	121.709999
19	8-9	(5, 5)	4.73	125.169998
20	9-10	(5, 5)	4.26	120.389999
21	10-11	(5, 5)	4.06	114.410004
22	11-12	(5, 5)	3.13	98.370003
23	12-13	(5, 5)	2.54	118.279999
24	1-2	(9, 9)	5.42	111.860001
25	2-3	(9, 9)	6.21	110.300003
26	3-4	(9, 9)	6.21	112.690002
27	4-5	(9, 9)	5.90	123.449997
28	5-6	(9, 9)	6.07	117.099998
29	6-7	(9, 9)	5.73	123.459999
30	7-8	(9, 9)	4.79	121.709999
31	8-9	(9, 9)	4.73	125.169998
32	9-10	(9, 9)	4.26	120.389999
33	10-11	(9, 9)	4.06	114.410004
34	11-12	(9, 9)	3.13	98.370003
35	12-13	(9, 9)	2.54	118.279999

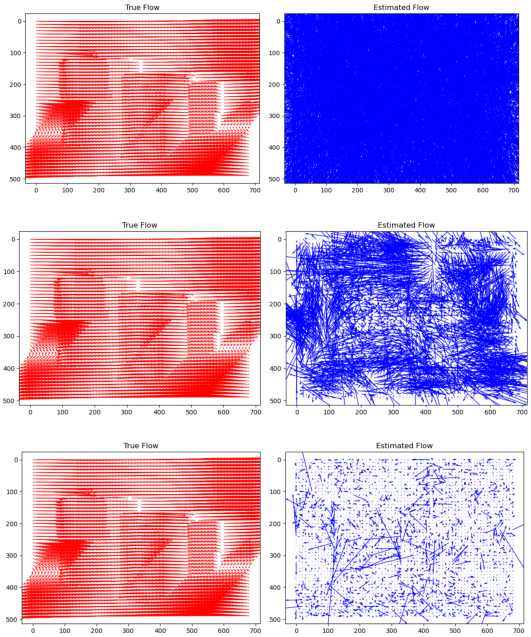
Appendix 2: Results from the LK (from scratch) on non -consecutive image frames 1 → 13



Appendix 3: Coarse-to-fine LK from frames 1 → 13 with (top) 2 layers and scaling factor of 1 and (bottom) 4 layers and scaling factor of 1



Appendix 4: The various estimated optical flow outputs on the quest for the Horn-Schunck tracker



Appendix 5: HS method optical flow evaluation for all 13 frames, with lambda values [0.001, 0.1, 1]

	frame	lambda	epe	ae
0	1-2	0.01	106.52	87.03
1	2-3	0.01	128.84	87.53
2	3-4	0.01	141.30	87.31
3	4-5	0.01	136.10	87.36
4	5-6	0.01	131.79	88.24
5	6-7	0.01	135.27	87.47
6	7-8	0.01	141.31	87.39
7	8-9	0.01	142.18	87.53
8	9-10	0.01	140.65	87.81
9	10-11	0.01	134.36	87.28
10	11-12	0.01	128.84	86.36
11	12-13	0.01	126.58	86.76
12	1-2	0.10	106.50	87.02
13	2-3	0.10	128.81	87.53
14	3-4	0.10	141.28	87.31
15	4-5	0.10	136.07	87.36
16	5-6	0.10	131.78	88.24
17	6-7	0.10	135.26	87.46
18	7-8	0.10	141.29	87.39
19	8-9	0.10	142.17	87.53
20	9-10	0.10	140.63	87.80
21	10-11	0.10	134.34	87.28
22	11-12	0.10	128.82	86.35
23	12-13	0.10	126.56	86.76
24	1-2	1.00	105.20	86.76
25	2-3	1.00	127.39	87.27
26	3-4	1.00	140.04	87.10
27	4-5	1.00	134.51	87.12
28	5-6	1.00	130.43	88.06
29	6-7	1.00	134.02	87.22
30	7-8	1.00	140.22	87.17
31	8-9	1.00	140.87	87.26
32	9-10	1.00	139.46	87.53
33	10-11	1.00	133.19	87.04
34	11-12	1.00	127.44	85.98
35	12-13	1.00	124.63	86.34

Appendix 6: Comparison of various optical flow estimation techniques in literature

Technique	Average Error	Standard Deviation	Density
Horn and Schunck (original)	32.43°	30.28°	100%
Horn and Schunck (original) $\|\|\nabla I\|\| \geq 5.0$	25.41°	28.14°	59.6%
Horn and Schunck (modified)	11.26°	16.41°	100%
Horn and Schunck (modified) $\|\|\nabla I\|\| \geq 5.0$	5.48°	10.41°	32.9%
Lucas and Kanade ($\lambda_2 \geq 1.0$)	4.10°	9.58°	35.1%
Lucas and Kanade ($\lambda_2 \geq 5.0$)	3.05°	7.31°	8.7%
Uras et al. (unthresholded)	10.44°	15.00°	100%
Uras et al. ($det(H) \geq 1.0$)	6.73°	16.01°	14.7%
Nagel	11.71°	10.59°	100%
Nagel $\|\|\nabla I\|\|_2 \geq 5.0$	6.03°	11.04°	32.9%
Anandan	15.84°	13.46°	100%
Singh (Step 1, $n = 2$, $w = 2$)	18.24°	17.02°	100%
Singh (Step 1, $n = 2$, $w = 2$, $\lambda_1 \leq 5.0$)	16.29°	25.70°	2.2%
Singh (Step 2, $n = 2$, $w = 2$)	13.16°	12.07°	100%
Singh (Step 2, $n = 2$, $w = 2$, $\lambda_1 \leq 0.1$)	12.90°	11.57°	97.8%
Heeger (combined)	11.74°	19.04°	44.8%
Heeger (level 0)	20.89°	34.26°	64.2%
Heeger (level 1)	10.51°	12.11°	15.2%
Heeger (level 2)	11.51°	11.83°	2.4%
Waxman et al. $\sigma_f = 2.0$	20.32°	20.60°	7.4%
Fleet and Jepson ($\tau = 1.25$)	4.95°	12.39°	30.6%
Fleet and Jepson ($\tau = 2.5$)	4.29°	11.24°	34.1%